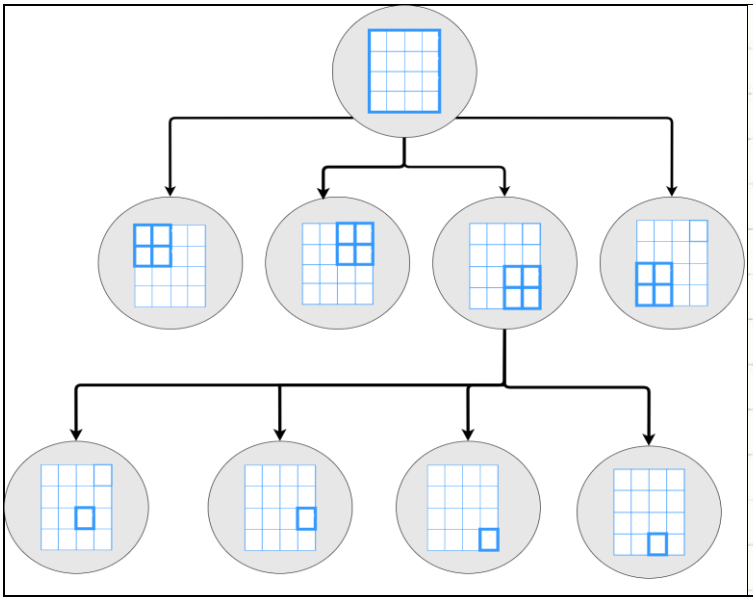
(بنية اللعبة)The Game Structure

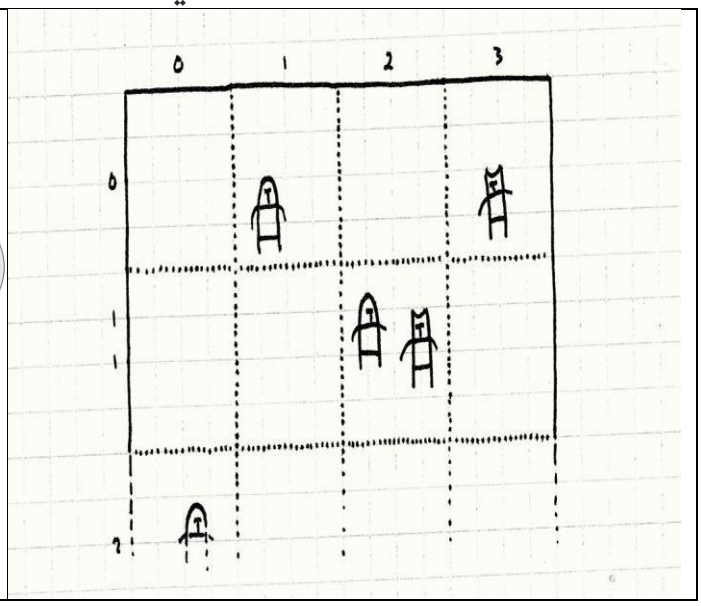
مقدمة

سنتحدث اليوم حول هياكل البيانات في تطوير الألعاب ..ليس لأن جميع الطلاب في السنة الرابعة يريدون ان يصبحوا مطوري ألعاب بل لأن هذه المادة تتطلب منا العمل على مشروع لعبة كجزء من المنهاج وهذه اول مرة لدى البعض يخوض فيها تجربة بناء لعبة كاملة لذا من الطبيعي ان تشعر بمزيج من الحماس وقليل من الخوف او التوتر فهذا شعور يمر به أي مطور في بداياته

لكن هذا المشروع فرصة رائعة ليس فقط لإكمال المادة بل لتطبيق الخوارزميات وهياكل البيانات على لعبة سنبنيها بأيدينا من الصفر ولذلك فإن اختيار هياكل بيانات مناسبة من البداية هو امر حاسم لضمان افضل أداء ممكن حتى لا تواجه مشكلة في السرعة او السلاسة عند تجريب الخوارزميات عليها .

تخيل بناء لعبة تحوي على العديد من الكائنات المتحركة وتصادمات فإذا اخترنا هيكل غير مناسب مثل مصفوفة بسيطة لعمليات تحتاج إلى إضافات وحذف متكرر سيؤدي هذا الاختيار إلى بطء الأداء بشكل ملحوظ مما يعيق تطبيق الخوارزميات بكفاءة أما الاختيار الصحيح في هذه الحالة هو ال(quad tree) لتقسيم المساحات فيحافظ على سلاسة اللعبة وإمكانية الحذف مما يسمح بأداء عالي.

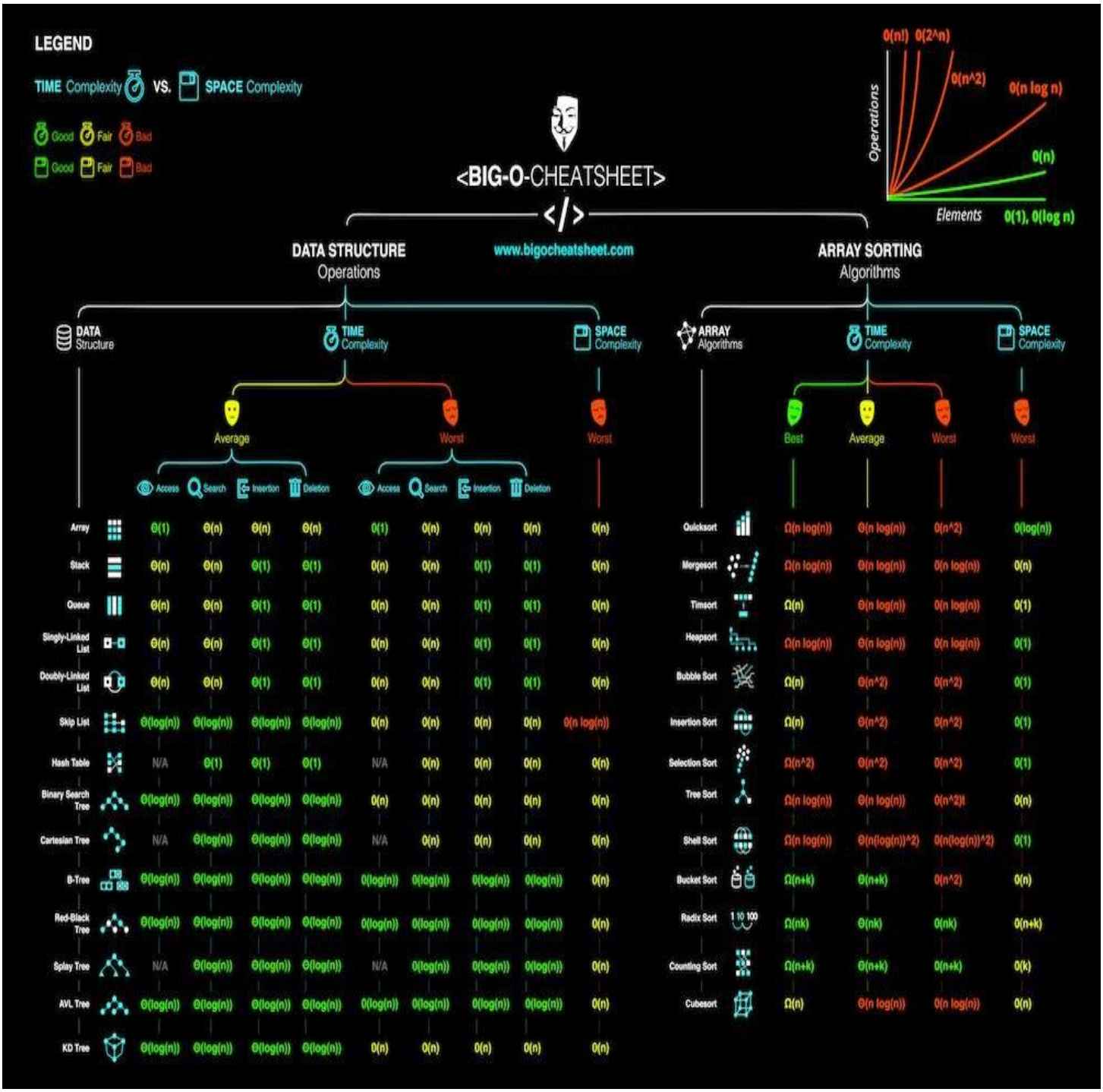




- لماذا يهمنا الوقت خلال اللعب؟

الألعاب هي أنظمة وقت حقيقي تتطلب أداء صارم واختيار هياكل البيانات السيئة يرفع التعقيد الزمني مثل O(n) في العمليات المتكررة بينما في الاختيارات الذكية يحافظ على O(1).

تذكرة بهياكل البيانات وخصائصهم:



- مما سبق نجد ان المفتاح هو الحذر في الاختيار حلل احتياجات لعبتك وما العمليات الأكثر تكرارا حجم البيانات ,ضع خطة واضحة من البداية .

Logic Maze Collapse

سنتحدث اليوم عن لعبة Logic Maze Collapse وكيف سنبني هيكلها بطريقة مناسبة .

اخترنا لعبة سهلة وبسيطة فيها نوع من التحدي فهي لعبة ألغاز منطقية ممتعة ومناسبة كثيرا للشرح لأننا سنطبقها باستخدام هياكل البيانات وخوارزميات البحث بشكل عملي وواضح.

ماهي طبيعة اللعبة بالضبط؟

يوجد لدينا شخصية صغيرة زرقاء (شكلها مثل هرم أو مكعب أزرق) تقف على شبكة من البلاطات معلقة بالفضاء. الهدف هو توصيل الشخصية لبوابة الخروج، لكن الامر لا يقتصر على عملية التوصيل... حيث يجب ان تمر الشخصية على كل بلاطة في المتاهة مرة واحدة فقط.

ما الذي يجعل اللعبة صعبة ومسلية:

لنتخيل أننا الشخصية بداخل اللعبة والآن لنقم باللعب، مع كل خطوة ننتقل فيها من بلاطةٍ إلى بلاطةٍ مجاورة (إلى اليمين، إلى اليسار، إلى الأعلى، أو إلى الأسفل)، تنهار البلاطة التي نتركها خلفنا وتختفي نهائيًا! لا يمكننا العودة إليها أبدًا. علينا أن نُفكِّر جيدًا في كل خطوة، حتى لا نحاصر أنفسنا في زاوية ونفقد القدرة على الوصول إلى البلاطات المتبقية. خطوة واحدة خاطئة، وسنضطر لإعادة المستوى من بدايته.

كيف نقوم باللعب:

1. نستخدم أسهم الكيبورد للتحريك (هو الأفضل للدقة).
2. نبدأ من مكان الشخصية، ونحاول إيجاد مسار يغطي كل الشبكة ويوصل للبوابة بالنهاية.
3. تحتوي اللعبة على مستويات عديده، تكون المستويات الأولى سهلة بهدف التعلم والتدرّب، ثم تزداد الصعوبة تدريجياً وتحتاج إلى تخطيط جاد ودقيق.

اللعبة ممتعة وتعلمنا الصبر والتفكير قبل التحرك، وبالوقت نفسه الرسومات فيها بسيطة، لكي نتمكن من برمجة نسخة مشابهة بسهولة نسبية في مشروعنا. بالنسبة لنا كطلاب تكنولوجيا المعلومات، فهي مثال ممتاز: يمكننا تمثيل المتاهة كـ رسمٍ بياني (Graph) واستخدام خوارزميات مثل البحث بعمق مع الرجوع (Backtracking) لإيجاد المسار الصحيح، أو استخدام مصفوفات لتمثيل الشبكة وطوابير للعمليات، ونُدرك أهمية اختيار الهيكل المناسب حتى يكون الأداء سريعًا حتى لو كبرت المتاهة.

لنقم بتجربتها قبل البدء بالبرمجة:

https://www.mathplayground.com/logic_maze_collapse_1.html

ستلاحظون مدى بساطة اللعبة من حيث المظهر الخارجي، مقابل عمقها وصعوبتها من حيث الفكرة والتحدي، وهو ما سيساعدنا على فهم الدرس بصورة أفضل :

ما هي طريقة تمثيل هذه اللعبة التي سنعتمد عليها

نبدأ أولًا بالتفكير في أهم الأصناف (Classes)التي يجب بناؤها، وما الخصائص الأساسية التي نحتاج إليها.

- أولاً:

سنقوم ببناء البنية الخاصة بنا بالاعتماد على نمد ال Model View Controller مع ملاحظة أننا لن نتطرق ل ال View لأن الهدف هنا هو محاكاة منطق اللعبة وحلها باستخدام الخوارزميات وليس عرضها بصريا.

إذا نحن بشكل أساسي امام:

- Model: يمثل حالة اللعبة وقوانينها.
- Controller: يمثل المنطق الذي يغير الحالة ويتحكم باللعبة.
- Playmode controller (ai search algorithm): يمثل العقل الذكي الذي يلعب اللعبة

- ثانياً :

ال Model هو تمثيل العالم الخاص باللعبة world representation

1. celltype class:

هذا الكلاس هو تجسيد لقوانين الخلايا الموجودة لدينا ,حيث أنواع الخلايا في اللعبة هي:

خلية عادية _ مفتاح _ قفل _ خلية قوية _ نهاية.

كل نوع خليه يحمل التالي (هذا موضح بالمشروع يمكنكم تحميله من الرمز في اخر المحاضرة):

- رمز هذه الخريطة.
- عدد الزيارات قبل السقوط.
- هل تحتاج مفتاح؟
- هل تعطي مفتاح؟

فائدة هذه الخطوة تأتي من عدم تكرار استخدام الشرط في كل مكان فقط نخزن القواعد داخل النوع نفسه

2. costtype class:

هذا الكلاس يحدد كلفة المرور فوق خلية معينة ,هو الوزن الموجود على الحواف في الرسم البياني ال Weighted graph ,وهو مهم من اجل خوارزميات مثل ال uniform cost search .

3. cell class :

الخلية تمثل ال : النوع الخاص بها ,عدد الزيارات لها ,هل سقطت؟ ,كم مفتاح فيها؟ وتكلفتها.

4. Maze:

تمثل المتاهة في كل لحظة حالة (State) من حالات اللعبة:

يمثل:

- أماكن كل الخلايا.
- مكان اللاعب(الهرم).
- التكلفة

نعتبر أن maze == State في خوارزميات البحث التي سوف نطبقها لاحقا ,ونتنبه ان يكون هذا الكلاس قابل للمقارنة والنسخ ولتخزين العقد التي تم زيارتها.

- ثالثاً :

ال controller وهو تمثيل لمنطق اللعبة.

1. CellController:

هذا المتحكم مسؤول عن:

- ماذا يحدث عند دخول خلية.
- ماذا يحدث عند مغادرتها.
- هل يمكن التحريك إليها أصلاً.

CellController = حكم المبارة.

هو لايعرف :

- المتاهة كاملة
- الخوارزمية
- الهدف النهائي

هو فقط يطبق قوانين الخلية نفسها .

2. MazeController:

منطق الحركة وتوليد الحالات ,هذا المتحكم هو قلب اللعبة, فهو مسؤول عن :

- تحولي الMaze → Grid .
- التحقق من صحة الحركة.
- توليد الحالات الممكنة.
- معرفة هل انتهت اللعبة.

MazeController = محرك القوانين العامة للعبة.

وهو:

- لايحل اللعبة

- لا يقرر أين يذهب
- فقط يقول: هذه هي كل الاحتمالات القانونية.

3. PlayModeController:

هذا المتحكم:

- لايعرف تفاصيل الخلايا.
- لايعرف كيف سقط.
- لايهتم بالقفل أو المفتاح.

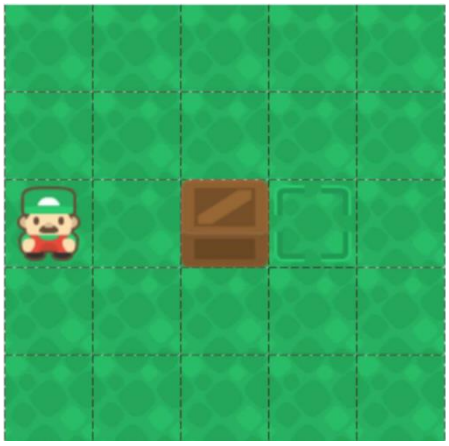
هو فقط يبحث عن طريق للوصول للنهاية بتطبيق خوارزمية معينه.

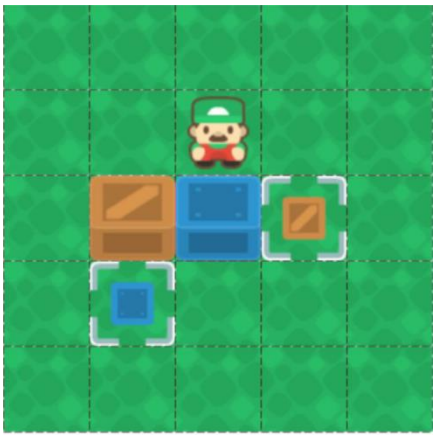
اللاعب الذكي = PlayModeController

- ملاحظة: الخوارزمية الموضحة بالكود هي Uniform Cost Search سيتم شرحها لاحقا
ملاحظة اربيسيزية : AI لا يحتاج فهم اللعبة، فقط يحتاج: حالة , أفعال , تكلفة , هدف

إضافة أربيسيزية:

- لعبة سوكمان :





لعبة Sokoban هي إحدى ألعاب الألغاز الكلاسيكية التي يلعبها لاعب واحد وتعتمد على المنطق، وهي تحقق برنامجًا بداية بكتابة برنامج يحقق هذه اللعبة.

يمكنك لعب اللعبة عبر الرابط: https://www.crazygames.com/game/sokoban

- قواعد اللعبة:

لدينا رقعة لعب أبعادها 5 × 5 (ممكن أن تكون أكبر أو مختلفة).
تحتوي على العامل، الصناديق (ممكن أن تكون بألوان مختلفة)، الحواجز، والخلايا المستهدفة (الهدف).
هناك فراغات (gaps) أو حفر في الرقعة.
يقوم العامل بالتحرك نحو الأعلى، الأسفل، اليمين، اليسار.
لا يمكن للعامل التحرك باتجاه معين عند وجود حاجز في ذلك الاتجاه.
لا يمكن للعامل التحرك باتجاه معين عند وجود صندوقين متتاليين في ذلك الاتجاه.

- الهدف من اللعبة هو وضع الصناديق في الخلايا المستهدفة حسب اللون.

- ملاحظات إضافية عن اللعبة:
- سوكوبان هي لعبة يابانية كلاسيكية من عام 1982، تتطلب دفع الصناديق إلى أماكن محددة (عادةً نقاط أو دوائر) دون سحبها، ولا يمكن دفع أكثر من صندوق واحد في المرة الواحدة.
- اللاعب (العامل) لا يستطيع المرور عبر الصناديق أو الجدران.

تحدّي أربيسيزي

- صمّموا عقل لعبة Sokoban قبل أن نبرمجها

اليوم لن نكتب كود اليوم سنفكّر كمهندسي برمجيات، لا كمبرمجين.

أنتم فريق تطوير طُلب منه بناء لعبة ألغاز كلاسيكية Sokoban :

✦ وصف اللعبة لدينا:

- رقعة لعب مربّعة (مثل 5×5، وقد تكبر لاحقًا)
- عامل (Player / Worker)
- صناديق (قد تكون بألوان مختلفة)
- دران أو حواجز
- خلايا هدف (Targets) مرتبطة بالألوان
- فراغات أو حفر لا تحتوي شيئًا

✦ قواعد أساسية:

- العامل يتحرك→ ← ↓ ↑
- لا يمكن المرور عبر الجدران أو الصناديق
- يمكن دفع صندوق واحد فقط في اتجاه الحركة

لا يمكن دفع صندوق إذا كان خلفه: جدار , صندوق آخر

الهدف: وضع كل صندوق في الخلية الهدف المناسبة له

- التحدّي الحقيقي (المطلوب منكم)

ممنوع التفكير بالكود الآن المطلوب منكم خلال 15–20 دقيقة:

اكتشاف الكلاسات الأساسية

اسألوا أنفسكم:

ما هي الكيانات (Entities) التي لا يمكن للعبة أن تعمل بدونها؟

ما الذي يمثل: حالة اللعبة؟ , اللاعب؟ ,الصندوق؟ ,الرقعة؟

تحديد مسؤولية كل كلاس (بكلمات بسيطة)

لكل كلاس:

ماذا يمثّل في العالم الحقيقي؟ , ماذا يجب أن يعرف؟ ,ماذا **لا يجب** أن يعرف؟

- مثال تفكيري (وليس حلاً):

هل الصندوق يعرف مكان الهدف؟ , أم أن هناك كلاس آخر يدير القواعد؟

التفكير بالتجريد(Abstraction)

ناقشوا:

- هل الحركة من مسؤولية العامل أم الرقعة؟
- هل القواعد داخل الصندوق أم داخل نظام يتحكم باللعبة؟
- هل نحتاج كلاس يمثّل حالة اللعبة كاملة؟

TQRCG

The End